

컴퓨터비전 개론 1주차 요약본

1. 딥러닝에 필요한 파이썬의 개념

1.1 개발 환경 설정

- **Anaconda란?**
 - 데이터 분석과 머신러닝을 위한 필수 라이브러리를 포함한 배포판
 - Pandas, NumPy, Matplotlib 등 주요 라이브러리 내장
 - Spyder, Jupyter Notebook 등 개발 도구(IDE) 제공
 - 버전 관리와 패키지 업데이트가 편리함
 - Windows, MacOS, Linux에서 모두 사용 가능
- **Anaconda 설치 과정**
 - [Anaconda 공식 홈페이지](#) 접속
 - 운영체제에 맞는 Anaconda 다운로드
 - 설치 후 환경 변수를 설정하여 conda 명령어를 사용할 수 있도록 함
- **Google Colab이란?**
 - Google에서 제공하는 무료 클라우드 기반 Jupyter Notebook 환경
 - 로컬 환경 설정 없이 머신러닝 및 딥러닝 개발 가능
 - GPU 및 TPU 지원
 - Google Drive와 연동하여 코드 및 데이터 저장 가능
- **Google Colab 접속 방법**
 - Google 계정으로 로그인
 - [Google Colab 접속](#) 후 "New Notebook" 선택
 - 런타임 유형을 GPU 또는 TPU로 변경 가능

1.2 딥러닝 개발 순서

- **문제 정의** – 해결할 목표 설정
- **데이터 수집** – 학습용 데이터 확보
- **전처리** – 데이터 정제 및 가공
- **모델 설계** – 모델 구조 및 설정 결정
- **학습** – 모델 훈련 및 파라미터 최적화
- **평가** – 성능 측정 및 검증
- **개선** – 하이퍼파라미터 조정 및 성능 향상
- **배포** – 운영 환경에 적용
- **모니터링** – 성능 추적 및 재학습 관리

1.3 Pytorch

- 페이스북이 개발한 **딥러닝 프레임워크**
- 직관적인 코드 작성 가능 → 연구 및 실무에서 많이 사용
- **동적 연산 그래프** 기반으로 디버깅과 실험이 쉬움
- GPU 연산 지원으로 빠른 학습 가능

- 행렬의 구조

```
A = np.array([[1, 2, 3], [4, 5, 6]])
print(A)
```

- 행렬의 연산

```
B = np.array([[7, 8, 9], [10, 11, 12]])
C = A + B # 행렬 덧셈
D = A - B # 행렬 뺄셈
```

-> 같은 크기의 행렬끼리만 덧셈/뺄셈 가능

```
E = np.dot(A, B.T) # A와 B의 전치 행렬을 곱함
```

-> 일반적인 행렬 곱셈을 수행할 때는 np.dot() 사용

-> 원소별 곱셈(Element-wise multiplication)은 A * B 사용

```
np.linalg.inv(A) # 역행렬
```

```
np.linalg.det(A) # 행렬식 계산
```

```
np.linalg.eig(A) # 고유값과 고유벡터 계산
-> 선형대수 연산 가능 (linalg 모듈 활용)
```

1.4 자동 미분과 수치 미분

구분	자동 미분 (Autograd)	수치 미분 (Numerical)
방식	연산 그래프 기반 기울기 계산	근사값 계산
정확도	매우 높음	h값에 따라 오차 존재
속도	빠름	느림
활용	학습 과정, 딥러닝 모델	검증, 디버깅

1.5 파이썬 객체 지향

- 클래스 (Class)
 - 객체를 만들기 위한 **설계도**
 - 속성과 메서드를 정의함.
- 인스턴스 (Instance)
 - 클래스로부터 생성된 **실제 객체**
- 속성 (Attribute)
 - 객체가 가지는 **데이터(상태)**
- 메서드 (Method)
 - 클래스 내부에 정의된 **함수**로, 객체의 동작을 정의

```
class Dog:
    def __init__(self, name):
        self.name = name # 속성
    def bark(self):      # 메서드
        print("멍멍!")

dog1 = Dog("바둑이") # Dog 클래스의 인스턴스
print(dog1.name) # '바둑이'
dog1.bark() # "멍멍!"
```

2. Numpy, Matplotlib

2.1 Numpy(Numerical Python)

- 행렬의 구조 및 연산
- 인덱싱 및 슬라이싱
- 행렬의 덧셈, 뺄셈, 곱셈, 전치(Transpose)
- Numpy 기본 구조

- 행렬의 구조

```
A = np.array([[1, 2, 3], [4, 5, 6]])  
print(A)
```

- 행렬의 연산

```
B = np.array([[7, 8, 9], [10, 11, 12]])
```

```
C = A + B # 행렬 덧셈
```

```
D = A - B # 행렬 뺄셈
```

-> 같은 크기의 행렬끼리만 덧셈/뺄셈 가능

```
E = np.dot(A, B.T) # A와 B의 전치 행렬을 곱함
```

-> 일반적인 행렬 곱셈을 수행할 때는 np.dot() 사용

-> 원소별 곱셈(Element-wise multiplication)은 A * B 사용

```
np.linalg.inv(A) # 역행렬
```

```
np.linalg.det(A) # 행렬식 계산
```

```
np.linalg.eig(A) # 고유값과 고유벡터 계산
```

-> 선형대수 연산 가능 (linalg 모듈 활용)

2.2 Numpy 속성 및 연산

- Numpy 배열 속성

```
array = np.array([[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9]], dtype=np.int32)
```

```
print(array)
```

```
print("array data type =", array.dtype) # 데이터 타입
print(array.shape) # 배열의 형태 (행, 열)
print(array.ndim) # 차원 개수
print(array.size) # 전체 요소 개수
```

- 특별한 행렬 생성

```
print(np.eye(2, 3)) # 단위 행렬 (대각선만 1, 나머지는 0)
print(np.identity(3)) # 정방형 단위 행렬
print(np.zeros([3,3])) # 0으로 채워진 행렬
print(np.ones((3,5))) # 1로 채워진 행렬
print(np.full((3,5), 5)) # 5로 채워진 행렬
```

단위 행렬 (eye, identity)
0으로 채운 행렬 (zeros)
1로 채운 행렬 (ones)
특정 값으로 채운 행렬 (full)

- Numpy 연산

```
x = np.arange(12) # 0~11까지 연속된 숫자 생성
y = np.linspace(0, 11, 12) # 0~11까지 12개 균등 분포 값 생성
arange(start, stop, step) → 정수 시퀀스 배열
linspace(start, stop, num) → 균등 간격으로 값 생성
```

```
x = np.arange(1, 7).reshape(2, 3)
print(x.T) # 전치행렬 (행과 열 교환)
```

```
np.matmul(x, x.T)
→ 행렬 곱 연산 (일반적인 곱셈)
```

```
x = np.array([[[0], [1], [2]]])
print(x.squeeze()) # 불필요한 차원 제거, 배열 차원 축소
.squeeze() → 차원이 1인 축을 제거
```

```
list1 = [[1, 2, 3]]
list2 = [[4, 5, 6]]
```

```
print(np.concatenate([list1, list2], axis=0)) # 행 방향 연결
print(np.concatenate([list1, list2], axis=1)) # 열 방향 연결
print(np.vstack((list1, list2))) # 행 방향 연결
print(np.hstack((list1, list2))) # 열 방향 연결
```

```
a = np.arange(4).reshape((2,2))
print(np.min(a)) # 전체 배열의 최소값
print(np.min(a, axis=0)) # 열 방향 최소값
print(np.max(a, axis=1)) # 행 방향 최대값
```

.min()/max() → 최소값/최대값 찾기 (axis=0: 열 기준, axis=1: 행 기준)

• 난수 생성

```
# 균등 분포 난수 (rand)
np.random.rand(3,2)
```

rand() → [0,1] 사이에서 랜덤 샘플링

```
# 정수 난수 (randint)
np.random.randint(1, 10, (3,2)) # 1~10 사이의 정수 난수
```

randint(low, high, size) → 정수 난수 생성

```
# 정규분포 난수 (randn)
np.random.seed(123) # 고정된 결과를 얻기 위한 시드 설정
np.random.randn(3,2) # 평균 0, 표준편차 1의 정규 분포 난수
randn() → 표준 정규 분포 (mean=0, std=1)에서 난수 샘플링
seed(n) → 랜덤 시드 설정 (결과 고정 가능)
```

2.3 Matplotlib (데이터 시각화)

• Matplotlib 구조

```
import matplotlib.pyplot as plt
```

```
x = np.linspace(-10, 10, 100) # -10부터 10까지 100개 점 생성
```

```

y = np.sin(x)

plt.plot(x, y)      # 선 그래프 그리기
plt.xlabel("X-axis") # x축 제목 설정
plt.ylabel("Y-axis") # y축 제목 설정
plt.title("Sine Wave") # 그래프 이름 설정
plt.grid()          # 격자 표시
plt.show()           # 그래프 출력

```

2.4 Matplotlib 활용

- 시그모이드 함수 그래프

```

# 시그모이드 함수 정의 및 그래프 그리기
def sigmoid(x, a):
    return 1/(1 + np.exp(-a*x))

xp = np.linspace(-3, 3, 61)
yp = sigmoid(xp, 1.0)
yp2 = sigmoid(xp, 2.0)

plt.plot(xp, yp)
plt.show()

시그모이드 함수 정의:  $\text{sigmoid}(x) = 1 / (1 + \exp(-a*x))$ 
xp → x값(-3 ~ 3 범위)
yp, yp2 → 시그모이드 함수 결과값

```

- Subplot(다수의 그래프 동시 출력)

```

# MNIST 데이터셋 시각화
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', version=1, as_frame=False)

image = mnist.data # 이미지 데이터
label = mnist.target # 정답 데이터

```

```
plt.figure(figsize=(10, 3))

for i in range(20):
    ax = plt.subplot(2, 10, i+1)
    img = image[i].reshape(28,28) # 28×28 픽셀로 변환
    ax.imshow(img, cmap='gray_r') # 흑백으로 출력
    ax.set_title(label[i]) # 타이틀(정답) 추가
    ax.set_xticks([]) # x축 눈금 제거
    ax.set_yticks([]) # y축 눈금 제거

plt.tight_layout() # 그래프 간격 조정
plt.show()
```

3. Pytorch의 기본 기능

3.1 PyTorch 개요

- **PyTorch란?**
 - 딥러닝 신경망 구축을 위한 오픈소스 프레임워크
 - Torch 머신러닝 라이브러리 + Python API를 결합
 - 쉽고 직관적인 문법을 제공하며, 동적 연산이 가능

- **PyTorch&TensorFlow 비교**

항목	Pytorch	Tensorflow
개발사	Meta(Pytorch Foundation)	Google
사용 분야	연구, 학계 등	산업용, 배품 제포 등
장점	사전 학습 모델과 문서가 많음	Google 서비스와 통합이 용이함

- **PyTorch 사용 현황**
 - GitHub 및 Hugging Face에서 활발하게 사용됨
 - 연구 커뮤니티에서 선호 및 TensorFlow보다 학습 용이성에서 강점
- **Pytorch 기본 구조 자료형**
 - **텐서 (Tensor)**

- PyTorch의 핵심 데이터 구조
- NumPy 배열과 유사하지만 GPU 가속이 가능
- 스칼라 (Scalar) — 0차원 텐서
- 벡터 (Vector) — 1차원 텐서
- 행렬 (Matrix) — 2차원 텐서
- 텐서 (Tensor) — 3차원 이상

```
import torch
# 스칼라 (0차원 텐서)
a = torch.tensor(5)
print(a.shape)
# 벡터 (1차원 텐서)
v = torch.tensor([1, 2, 3])
print(v.shape)
# 행렬 (2차원 텐서)
m = torch.tensor([[1, 2], [3, 4]])
print(m.shape)
# 3차원 텐서 (예: 배치 x 행 x 열)
t = torch.randn(2, 3, 4)
print(t.shape)
```

○ PyTorch 자료형

- torch.FloatTensor → 32비트 실수형 텐서
- torch.IntTensor → 32비트 정수형 텐서
- torch.cuda.FloatTensor → GPU 가속된 실수형 텐서

3.2 PyTorch를 활용한 미분 및 경사 계산

```
# Pytorch 텐서를 Numpy로 변환
r2_np = r2.data.numpy()
print(type(r2_np)) # NumPy 배열 타입 확인
print(r2_np) # 변환된 값 출력
PyTorch 텐서를 NumPy 배열로 변환 (.data.numpy())
CPU 상에서 NumPy 연산과 PyTorch 연산을 혼합하여 사용할 수 있음
```

#2차 함수의 경사(미분) 계산

`x_np = np.arange(-2, 2.1, 0.25)` # -2부터 2까지 0.25 간격의 NumPy 배열 생성

`x = torch.tensor(x_np, requires_grad=True, dtype=torch.float32)`

→ `requires_grad=True`

→ PyTorch가 자동으로 미분(Gradient)을 계산할 수 있도록 설정

2차 함수 $y = 2x^2 + 2$ 을 정의

`y = 2 * x**2 + 2`

`print(y)`

계산 그래프 자동 생성 및 시각화

`z = y.sum()` # 경사 계산을 위해 `sum()` 사용

`from torchviz import make_dot`

`g = make_dot(z, params={'x': x})` # 계산 그래프 시각화

`display(g)`

PyTorch 내부에서 자동으로 미분 연산을 위한 계산 그래프 생성

torchviz를 활용하여 그래프를 시각화

3.3 자동 미분(Autograd)으로 경사 계산

경사(Gradient) 계산

`z.backward()` # 자동 미분 수행

`print(x.grad)` # x에 대한 미분값 출력

`backward()`를 호출하면 `requires_grad=True`가 설정된 텐서의 미분값이 자동으로 계산

출력된 값은 2차 함수 $y = 2x^2 + 2$ 의 도함수인 $dy/dx = 4x$

함수(y)와 미분값(Gradient) 그래프를 동시에 출력하여 변화율 시각화

`plt.plot(x.data, y.data, c='b', label='y')` # 함수 그래프

`plt.plot(x.data, x.grad.data, c='k', label='y.grad')` # 경사 그래프

`plt.legend()`

`plt.show()`

경사를 초기화하지 않으면 값이 누적

```

y = 2 * x**2 + 2
z = y.sum()
z.backward()
print(x.grad)
backward()를 두 번 실행하면 경사 값이 누적됨

# 경사 초기화 (zero_())
x.grad.zero_()
print(x.grad)
zero_()를 사용하여 경사값을 초기화
PyTorch는 기본적으로 경사값을 누적하므로, 매번 초기화하는 것이 중요

```

3.4 Pytorch 활용하여 시그모이드 함수와 자동 미분(Gradient Calculation)

- 시그모이드 함수 정의 및 계산

```

# PyTorch 내장 Sigmoid 함수 사용
sigmoid = torch.nn.Sigmoid()
y = sigmoid(x)
PyTorch의 torch.nn.Sigmoid()를 이용하여 시그모이드 함수 정의
입력 x에 대한 y = sigmoid(x) 값 계산

# 스칼라 값으로 변환
z = y.sum() # 자동 미분을 수행하려면 최종 값이 스칼라여야 함
PyTorch의 자동 미분(Autograd)을 사용하려면,
최종 결과를 하나의 스칼라 값으로 변환해야 함
여기서는 sum()을 사용하여 모든 값을 하나의 값으로 합산

# 계산 그래프 시각화
g = make_dot(z, params={'x': x})
display(g)
torchviz를 사용하여 계산 그래프 시각화

```

- 자동 미분(Gradient Calculation)

```
# 미분(경사) 계산
z.backward()
print(x.grad) # x에 대한 경사(미분값) 출력
backward()를 호출하면 PyTorch가 자동으로 미분을 계산
출력된 값은 dy/dx로, 시그모이드 함수의 도함수를 나타냄

# 경사값 초기화 (zero_())
x.grad.zero_()
print(x.grad)
PyTorch는 기본적으로 경사값을 누적하므로,
새로운 계산을 위해 zero_()를 사용하여 초기화해야 함
```

• 사용자 정의 시그모이드 함수 구현

```
# PyTorch 내장 함수 없이 직접 시그모이드 함수 구현
def sigmoid(x):
    return 1 / (1 + torch.exp(-x))

# 사용자 정의 함수에 대한 그래디언트 계산
y = sigmoid(x)
→ 정의한 sigmoid(x)를 사용하여 y값을 계산

z = y.sum() # 스칼라 값으로 변환
params = {'x': x}
g = make_dot(z, params=params)
display(g)

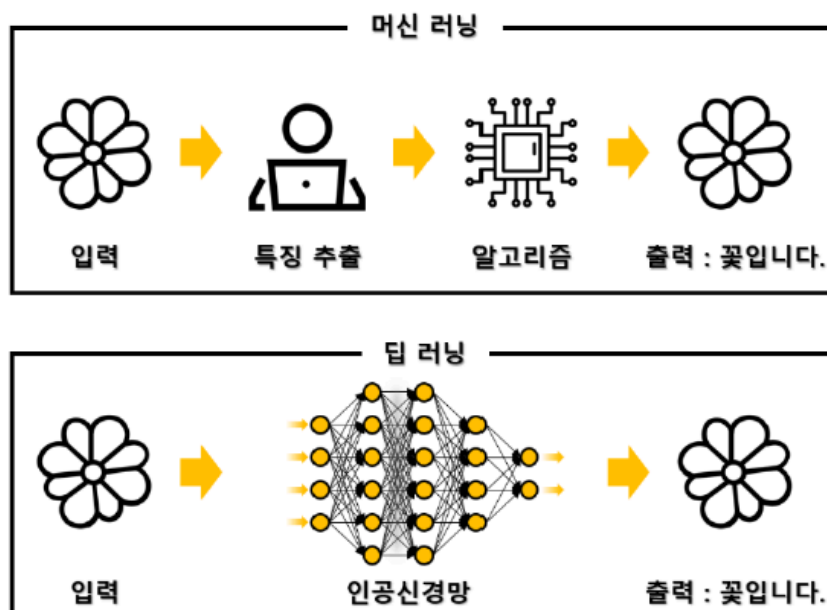
z.backward() # 자동 미분 수행
print(x.grad) # 경사 출력
```

4. 인공지능 개론

4.1 AI, 머신러닝(ML), 딥러닝(DL) 개요

- AI (Artificial Intelligence, 인공지능)
 - 인간의 사고를 모방하는 기술

- 규칙 기반 시스템부터 최신 딥러닝 모델까지 포함
- **ML (Machine Learning, 머신러닝)**
 - AI의 하위 분야로, 데이터를 이용해 패턴을 학습하고 예측하는 기술
 - 지도학습, 비지도학습, 강화학습 등의 방법 포함
- **DL (Deep Learning, 딥러닝)**
 - 머신러닝의 한 분야로, 다층 신경망(Neural Network)을 이용한 학습 기법
 - 데이터 양이 많을수록 성능이 향상됨
- 머신러닝의 종류
 - 지도학습(Supervised Learning)
 - 문제와 정답 모두 알려주고 학습 → 예측(prediction), 분류(Classification)
 - 비지도 학습(Unsupervised Learning)
 - 답을 가르쳐주지 않고 공부시키는 방법 → 연관성 파악, 군집(Clustering)
 - 강화 학습(Reinforcement Learning)
 - 보상을 통해 상은 최대화, 벌은 최소화하는 방향으로 행위를 강화하는 학습 → 보상
- 머신러닝과 딥러닝 비교
 - 머신러닝(ML)은 특징을 사람이 직접 추출해야 함
 - 딥러닝(DL)은 특징 추출 과정도 신경망이 자동으로 수행



- 데이터 세트
 - 훈련 데이터 (Training Data) : 모델을 학습시키는 데 사용하는 데이터
 - 검증 데이터 (Validation Data) : 학습 중 모델의 성능을 점검하는 데이터
 - 테스트 데이터 (Test Data) : 최종 모델의 성능을 평가하는 데이터
- 모델 성능 평가 지표
 - 회귀

지표	의미	특징
MAE (Mean Absolute Error)	실제값과 예측값의 절대값 오차의 평균	모든 오차를 동일하게 취급, 해석이 직관적
RMSE (Root Mean Squared Error)	오차 제곱의 평균에 제곱근을 취한 값	큰 오차에 더 민감, 이상치 영향 큼
R² (결정계수)	모델이 데이터를 얼마나 잘 설명하는가를 나타냄	1에 가까울수록 설명력이 높음, 0 이하는 매우 부적합

- 분류

지표	의미	계산식/특징
정확도 (Accuracy)	전체 중 맞게 예측한 비율	$(TP + TN) / (TP + TN + FP + FN)$
정밀도 (Precision)	모델이 '양성'이라고 예측한 것 중 실제로 양성인 비율	$TP / (TP + FP)$
재현율 (Recall)	실제 양성 중 모델이 '양성'이라고 맞춘 비율	$TP / (TP + FN)$
F1-score	정밀도와 재현율의 조화평균	$2 \times (Precision \times Recall) / (Precision + Recall)$
ROC-AUC (Area Under Curve)	분류 임계값에 따라 성능을 평가한 곡선의 면적	1에 가까울수록 좋음, 클래스 불균형에도 강함

4.2 선형회귀와 경사 하강법

- 선형 회귀 (Linear Regression)
 - 입력 x 와 출력 y 의 관계를 직선으로 모델링
 - 수식 : $y = wx + b$
- 경사 하강법 (Gradient Descent)
 - 손실 함수를 가장 작게 만드는 방향으로 w , b 를 반복 조정

4.3 신경망과 퍼셉트론(Perceptron)

- **신경세포와 퍼셉트론**
 - 인공 뉴런(Perceptron)은 생물학적 신경세포를 모방하여 개발됨
 - 뉴런이 여러 개 연결되어 네트워크를 형성하면 딥러닝 모델이 됨
- **퍼셉트론 구조**
 - 입력층(Input layer) → 가중치(Weight)와 바이어스(Bias) 적용 → 활성화 함수(Activation Function) → 출력층(Output layer)

4.4 XOR 문제와 다층 신경망

- **퍼셉트론의 한계**
 - 단층 퍼셉트론은 선형 분리가 가능한 문제만 해결 가능
 - XOR 문제는 단층 퍼셉트론으로 해결할 수 없음
- **다층 신경망(MLP, Multi-Layer Perceptron)**
 - 은닉층(Hidden Layer)을 추가하여 XOR 문제 해결 가능
 - 입력층 → 은닉층 → 출력층 구조를 가짐

4.5 신경망 학습과 오류 역전파 (Backpropagation)

- **신경망 학습 과정**
 - **순전파(Forward Propagation)**
 - 입력 데이터를 신경망을 통해 전달하여 예측값 출력
 - **손실 함수(Loss Function) 계산**
 - 예측값과 실제값의 차이를 계산
 - **역전파(Backpropagation)**
 - 손실을 최소화하기 위해 가중치와 바이어스를 조정
 - 체인 룰(Chain Rule)을 이용하여 미분
 - **가중치 업데이트**

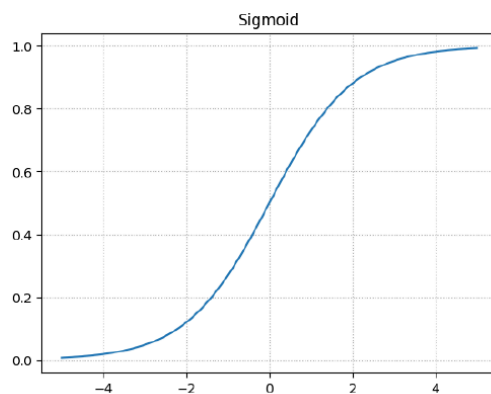
- 경사 하강법(Gradient Descent) 또는 Adam 옵티마이저(Optimizer)로 최적화 수행
- 기울기 소실(Vanishing Gradient) 문제
 - 문제 발생 원인
 - 역전파 과정에서 기울기(Gradient)가 점점 작아져서 학습이 제대로 진행되지 않는 현상
 - 주로 시그모이드 함수나 tanh 함수에서 발생
 - 해결 방법
 - ReLU(Rectified Linear Unit) 활성화 함수 사용
 - 가중치 초기화 방법 개선 (Xavier, He Initialization 등)
 - Batch Normalization 사용

4.6 활성화 함수(Activation Function)

- Sigmoid

$$\delta(x) = \frac{1}{1 + e^x}$$

- 출력 값을 0에서 1
- 가장 오래된 함수
- Saturation (기울기 소실)
- Not zero-centered

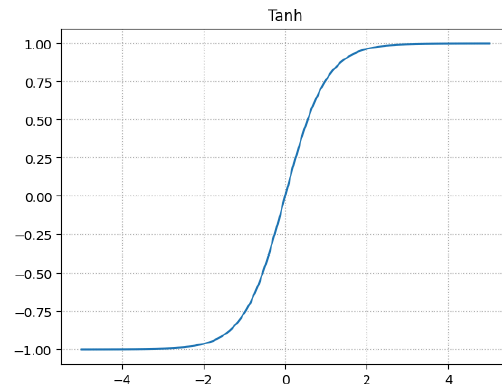


→ 출력값이 (0,1) 사이, 기울기 소실 문제 발생 가능

- Tanh

$$\delta(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

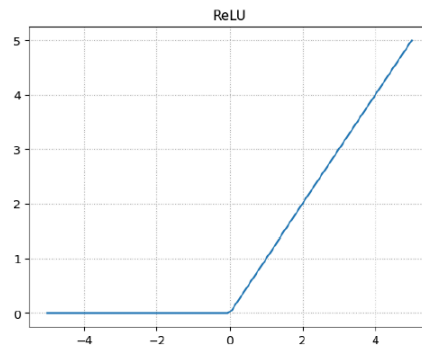
- 출력 값은 -1에서 1
- Still Saturation
- Zero-centered



• ReLU

$$\delta(x) = \max(0, x)$$

- 출력 값을 0에서 ∞
- 계산 효율이 뛰어남. sigmoid보다 훨씬 빠름
- 양의 구간에서 Not Saturated

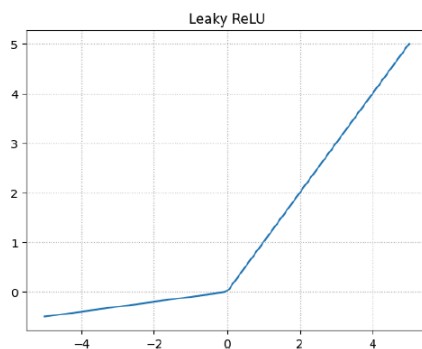


→ 기울기 소실 문제 해결, 빠르고 효율적

• Leaky ReLU

$$\delta(x) = \begin{cases} x, & \text{if } x \geq 0 \\ ax, & \text{else} \end{cases}$$

- 음수구간에서 0이 아님
- 계산 효율이 뛰어남. sigmoid보다 훨씬 빠름
- Not Saturated



→ ReLU의 문제 해결, 음수 구간에서도 기울기 존재

4.7 최적화 함수 (Optimizer)

- 신경망의 가중치(weight)를 업데이트하여 손실(loss)을 최소화하는 알고리즘

Optimizer	특징
SGD (Stochastic Gradient Descent)	기본 형태의 경사 하강법. 단순하지만 수렴 속도가 느릴 수 있음
Momentum	이전 이동 방향을 고려해 진동을 줄이고 빠른 수렴을 유도
AdaGrad	학습률을 변수마다 다르게 조정. 드물게 등장하는 변수에 더 큰 학습률 부여
RMSProp	AdaGrad의 단점을 보완. 최근 기울기에 가중치를 두어 학습률 감소 문제 해결
Adam (Adaptive Moment Estimation)	Momentum + RMSProp 결합형. 가장 널리 사용되는 최적화 알고리즘
AdamW	Adam에 정규화(weight decay)를 추가해 과적합 방지

5. 인공지능 학습

5.1 합성 함수

- 합성 함수란?
 - 두 함수 $f(x)$ 와 $g(x)$ 가 있을 때, 한 함수의 출력값을 다른 함수의 입력값으로 사용하는 방식
 - 수학적 표현 및 예제

$$h(x) = f(g(x))$$

$$f(x) = 2x + 3, \quad g(x) = x^2$$

$$z(x) = f(g(x)) = 2x^2 + 3$$

- 딥러닝에서는 여러 개의 층을 쌓아, 합성 함수를 구성하여 복잡한 문제를 해결

5.2 경사 하강법(Gradient Descent)

- 경사 하강법이란?
 - 머신러닝 및 최적화 문제에서 널리 사용되는 알고리즘
 - 비용 함수(Loss Function)를 최소화하는 방향으로 가중치를 업데이트

- 신경망 학습 과정에서 필수적인 최적화 기법

• 경사 하강법의 단계

1. 초기값 설정: 파라미터(가중치)를 임의로 초기화
2. 기울기 계산: 현재 위치에서 함수의 기울기(미분값)를 계산
3. 파라미터 업데이트: 기울기의 반대 방향으로 이동하여 최적값을 찾음
4. 반복: 수렴할 때까지 위 과정을 반복

$$\theta_{n+1} = \theta_n - \alpha \nabla_{\theta} J(\theta_n)$$

θ : 파라미터 (예: 가중치).

α : 학습률 (learning rate).

$\nabla_{\theta} J(\theta)$: 비용 함수 $J(\theta)$ 의 기울기.

• 경사 하강법의 원리

→ 기울기 방향을 따라 내려가면서 손실을 줄이는 과정

• 학습률 (lr: Learning Rate)

- 학습률 α 값이 너무 크면 **Overshooting** (최적점을 지나침) 발생
- 학습률 α 값이 너무 작으면 수렴 속도가 느려짐

• 경사 하강법의 종류

◦ 배치 경사 하강법 (Batch Gradient Descent)

- 전체 훈련 데이터를 한 번에 사용하여 기울기를 계산
- 장점: 전체 데이터를 사용하여 안정적인 학습 가능
- 단점: 데이터가 많을 경우 계산 비용이 큼

◦ 확률적 경사 하강법 (Stochastic Gradient Descent, SGD)

- 훈련 데이터에서 하나의 샘플을 랜덤하게 선택하여 기울기를 계산
- 장점: 계산 속도가 빠르고, 메모리 요구량이 적음

- 단점: 매 스텝마다 최적화 방향이 바뀌어 학습이 불안정할 수 있음
- **미니배치 경사 하강법 (Mini-batch Gradient Descent)**
 - 훈련 데이터를 작은 배치 단위로 나누어 각 배치에서 기울기를 계산
 - 장점: 배치 경사 하강법의 안정성과 SGD의 속도를 조합
 - 단점: 배치 크기 선택에 따라 성능이 달라질 수 있음

5.3 학습 순서

단계	내용	PyTorch 예시 코드
1. 예측	입력을 모델에 넣어 예측값 계산	<code>outputs = net(inputs)</code>
2. 손실 계산	예측값과 실제값 비교	<code>loss = criterion(outputs, labels)</code>
3. 역전파	손실에 따른 기울기 계산	<code>loss.backward()</code>
4. 최적화	가중치 업데이트	<code>optimizer.step()</code>

5.4 학습 스케줄러

- 스케줄러의 종류

스케줄러	특징
StepLR	일정 에폭마다 학습률을 일정 비율로 감소시킴
ExponentialLR	학습률을 매 스텝마다 지수적으로 감소시킴
MultiStepLR	특정 에폭 구간마다 감소시킴
CosineAnnealingLR	학습률을 코사인 곡선 형태로 감소시켜 부드럽게 수렴
ReduceLROnPlateau	손실이 일정 기간 개선되지 않을 때 학습률을 줄임 (적응형 방식)
OneCycleLR	학습 중 학습률을 증가 후 감소시키는 전략 (빠른 수렴에 효과적)

- **너무 큰 학습률:** 발산하거나 최소점을 지나침
- **너무 작은 학습률:** 학습 속도 느림, 지역 최소점에 머무를 수 있음

5.5 배치 (Batch)

- 배치는 한 번에 모델에 넣어 학습시키는 데이터 묶음이다.
- 전체 데이터를 한 번에 학습시키지 않고, 여러 묶음으로 나누어 처리한다.
- Full Batch (전체 배치 학습)

- 한 번의 학습(가중치 업데이트)에 **전체 데이터셋을 모두 사용**
 - Mini Batch (미니배치 학습)
 - 데이터를 작은 묶음(예: 32, 64개 등)으로 나누어 학습
 - 배치 정규화 (Batch Normalization)
 - 배치 단위로 데이터를 정규화하여 학습 안정성과 속도를 높이는 기법
 - 내부 공변량 변화(Internal Covariate Shift)를 줄이는 데 도움을 줌
-